

Lecture 5b. Ensemble Methods and Boosted Trees for Classification

Data mining, prof. Biljana Tojtovska Ribarski
FCSE, Skopje

Contents

1	Ensemble Methods and Boosted Trees for Classification	1
1.1	From a Single Tree to an Ensemble	1
1.2	Why Single Trees Can Be Unstable?	2
1.3	The General Idea of Ensemble Learning	3
1.4	Bagging: Averaging Many Trees	3
1.5	Out-of-Bag Error	5
1.6	Random Forest	6
1.7	Measuring Feature Importance	7
1.8	Boosting: Sequential Error Correction	8
1.9	AdaBoost	9
1.10	Gradient Boosting for Classification	10
2	Modern Boosted Tree Systems	16
2.1	Common backbone	16
2.2	Design differences	16
2.3	XGBoost	16
2.4	LightGBM	17
2.5	CatBoost	18
2.6	Practical Hyperparameters and Their Meaning	18
2.7	Practical Do's and Don'ts for Modern Boosted Tree Systems	19
2.8	Compact Comparison of Tree-Based Classification Methods	20
2.9	Summary	20

1 Ensemble Methods and Boosted Trees for Classification

1.1 From a Single Tree to an Ensemble

In the previous lectures, decision trees were introduced as intuitive classification models: they partition the feature space through a sequence of split conditions and produce a class decision at the leaves. They are attractive because they are easy to read, easy to visualize, and naturally handle nonlinear interactions.

But the previous lectures also established three important ideas:

- a model should be judged by its performance on unseen data, not by training fit alone;
- classifier quality depends on the practical goal, the class distribution, and the consequences of errors;

- a tree that looks convincing on one sample may change noticeably if the sample changes.

Next we want to understand what happens when many trees are combined into one stronger classifier.

Insight 1.1: Why this is the natural next step?

A single tree is often attractive because of interpretability, but it may be unstable and may overfit. Ensemble methods are a natural continuation because they try to keep the flexibility of tree-based learning while improving stability and predictive performance.

1.2 Why Single Trees Can Be Unstable?

A decision tree is built greedily: at each node, the algorithm searches for a split that looks best according to some impurity criterion such as entropy, Gini index, or misclassification error. Small changes in the training data may change which split looks best. Once an early split changes, the whole lower part of the tree may change as well.

Definition 1.1: Decision tree instability

A decision tree is called *unstable* if small perturbations in the training data can produce a noticeably different fitted tree and therefore different predictions.

Example 1.1: Medical triage threshold instability

Suppose a hospital uses a decision tree at triage to classify patients into **Urgent respiratory condition** versus **Non-urgent other condition**. If one or two patients close to a threshold on oxygen saturation or symptom severity are added, removed, or corrected, the first split of the tree may change. Then the entire diagnostic structure below that split may change as well.

This instability is closely related to the bias-variance tradeoff. A shallow tree is often too simple and may have high bias. A very deep tree may capture detailed local structure, but it may also become too sensitive to accidental patterns in the training data, which leads to high variance.

Definition 1.2: Bias-variance tradeoff

The **bias-variance tradeoff** describes two competing sources of prediction error:

- *bias*: error that occurs when a model is too rigid or too simple to capture the true underlying pattern in the data well. Such a model makes restrictive assumptions and can systematically produce inaccurate predictions.
- *variance*: error that occurs when a model depends too strongly on the particular training sample. In that case, small changes in the training data can lead to a noticeably different fitted model and different predictions.

A good model should be flexible enough to capture the real structure in the data, but stable enough that small changes in the training sample do not lead to large changes in prediction.

Remark 1.1: Why this matters?

Instability is not automatically bad. In fact, it becomes useful in ensemble learning. If different training perturbations produce different trees, then those trees may make different errors, and combining them can improve generalization.

1.3 The General Idea of Ensemble Learning

An **ensemble** combines multiple base models into one final predictor. For classification, the combination is often made by majority vote or by aggregating scores or probabilities.

When the base learners are trees, two broad ensemble philosophies dominate:

1. **Bagging**: build many trees independently on perturbed versions of the training data and aggregate their predictions.
2. **Boosting**: build trees sequentially so that each new tree improves what the current ensemble is still getting wrong.

Definition 1.3: Ensemble method

An **ensemble method** is a learning strategy that combines several base models into one final predictor, usually to improve stability, robustness, or predictive accuracy.

Remark 1.2: Two fundamentally different ideas

Although bagging and boosting both combine many trees, they should not be treated as the same idea:

- bagging mainly reduces variance through averaging;
- boosting mainly improves predictive fit through sequential correction.

1.4 Bagging: Averaging Many Trees

1.4.1 Bootstrap sampling

Bagging begins with the bootstrap. A bootstrap sample is obtained by sampling, with replacement, from the training set until one again has a sample of the same nominal size.

Definition 1.4: Bootstrap sample

Given a dataset of size n , a **bootstrap sample** is obtained by drawing n observations with replacement from the original dataset.

Sampling *with replacement* means that after an observation is drawn, it is placed back and can be drawn again. Thus one bootstrap sample may contain repeated copies of some observations, while others may not appear at all. This repetition is not an error; it is the central mechanism of the bootstrap.

Example 1.2: Bootstrap example

If the training set is

$$\{A, B, C, D, E\},$$

then a valid bootstrap sample of size 5 could be

$$\{B, B, D, E, E\}.$$

Here, B and E appear twice, while A and C are absent.

Remark 1.3: What bootstrap resampling means in practice?

Bootstrap resampling is a way to mimic repeated sampling from the population by using the observed dataset as an empirical stand-in for that population. If a train/validation/test

split already exists, bootstrap resampling is applied to the *training set*, while the validation and test sets remain untouched.

Warning 1.1: Bootstrap is not a universal fix

Bootstrap resampling does not automatically preserve every important structure. If the data are time-dependent, clustered, severely imbalanced, or already contaminated by leakage, then ordinary bootstrap may reproduce the wrong structure. The resampling unit must match the data-generating structure. (In such cases, one uses adapted resampling schemes, such as block bootstrap for time-dependent data, cluster-level resampling for grouped data, or stratified resampling when class proportions must be preserved.)

1.4.2 Bagging algorithm

Algorithm 1.1: Bagging for classification

Let the training set be

$$\mathcal{D} = \{(x_i, y_i) | i \in \{1, 2, \dots, n\}\}$$

Choose the number of trees B .

For $b = 1, \dots, B$:

1. draw a bootstrap sample $\mathcal{D}^{(b)}$ from $\mathcal{D}$;
2. train a decision tree T_b on $\mathcal{D}^{(b)}$.

For a new unseen observation x , obtain predictions $T_1(x), \dots, T_B(x)$ and predict the class by majority vote.

For classification, bagging may combine the trees either by majority vote on the predicted class labels or by averaging class scores/probabilities and then assigning the class using a chosen threshold.

1.4.3 Why bagging works?

Bagging is most effective when the base learner is unstable. Different bootstrap samples produce somewhat different trees. If those trees do not make exactly the same errors, averaging or voting can reduce the net effect of sample-specific noise.

Insight 1.2: Bagging as variance reduction

Bagging is best understood as a variance-reduction method. It does not try to learn one perfect tree. Instead, it averages many unstable trees so that random fluctuations of individual trees matter less.

Remark 1.4: Why the correlation of errors matters?

Each tree's prediction error can be viewed as having two parts: a part coming from the real difficulty of the problem, and a part coming from random sample-specific noise, that is, from accidental patterns present in the particular bootstrap sample on which that tree was trained. The role of bagging is mainly to reduce this second part.

The key question is whether different trees make *similar* random errors or *different* random errors. This is captured by the correlation ρ between their errors. If ρ is high, the trees tend to be wrong on the same cases in the same way, so averaging helps little. For example, if for one case the true target is 0, and three trees predict 0.9, 0.85, and 0.88, then all three are making almost the same error, so their average is still about 0.877, which remains far

from the truth. In contrast, if the three trees predict 0.9, 0.2, and 0.1, then their errors are less aligned, and their average is

$$\frac{0.9 + 0.2 + 0.1}{3} = 0.4,$$

which is noticeably closer to the true target. This is what it means that averaging can cancel part of the noise.

In simplified form, if each tree has error variance σ^2 and the pairwise error correlation is ρ , then the variance of the average of B trees is approximately

$$\text{Var}(\bar{f}) \approx \sigma^2 \left(\rho + \frac{1 - \rho}{B} \right).$$

Thus, when ρ is smaller, averaging many trees reduces variance much more; when ρ is close to 1, the trees behave too similarly, so bagging has much less effect.

1.4.4 What bagging does and does not do?

Bagging can improve predictive stability, but it does not solve every problem.

- It does not repair data leakage.
- It does not guarantee interpretability.
- It does not eliminate the need for validation.
- It does not fix a poor label definition or a poor feature set.

1.5 Out-of-Bag Error

When a tree is trained on a bootstrap sample, not every training observation is selected into that sample. The observations that are left out can still be used in a useful way: they act like a small temporary test set for that particular tree. These omitted observations are called *out-of-bag* observations.

Definition 1.5: Out-of-bag observation

For a given bootstrap sample, an observation is *out-of-bag* if it was not included in that sample.

Now consider one training observation. Some trees were trained using it, but some trees did not see it at all because it was out-of-bag for them. We can therefore predict its class using only the trees that did not use it during training. If we do this for all training observations and compare the aggregated predictions to the true classes, we obtain the *out-of-bag error*.

Remark 1.5: Why OOB is useful?

Out-of-bag error is especially useful in bagging and Random Forest because it provides a built-in estimate of predictive performance as part of the training procedure itself. This allows evaluation without creating an extra validation split inside the training data, and often with less computational cost than repeated cross-validation.

Warning 1.2: What OOB does not replace?

Out-of-bag error is helpful, but it does not replace a final untouched test set. Final model assessment should still be based on honest evaluation on data not used in model building.

1.6 Random Forest

1.6.1 Why plain bagging is not always enough?

If every bagged tree sees all features at every split, many trees may become too similar, especially when one variable is very dominant. In that case, averaging many similar trees brings less benefit than averaging more diverse trees.

1.6.2 The Random Forest idea

Random Forest extends bagging by adding *feature randomness*. Each tree is trained on a bootstrap sample, but at each split the algorithm considers only a random subset of candidate features.

Definition 1.6: Random Forest

A *Random Forest* is an ensemble of decision trees in which:

- each tree is trained on a bootstrap sample of the data;
- each split is chosen from a random subset of features rather than from all features;
- final prediction is obtained by majority vote in classification.

This produces two sources of randomness:

- observation-level randomness through bootstrapping;
- feature-level randomness through random candidate subsets.

The purpose of the second source is to reduce correlation among the trees, so that averaging becomes more effective. Here, and in the rest of the lecture notes, "averaging" does not mean averaging the trees themselves. It means averaging the predictions they produce, such as class scores or estimated probabilities. If the trees output only class labels, the ensemble prediction is usually obtained by majority vote.

Algorithm 1.2: Random Forest for classification

Let the training set be

$$\mathcal{D} = \{(x_i, y_i) | i \in \{1, 2, \dots, n\}\}$$

Choose the number of trees B .

For $b = 1, \dots, B$:

1. draw a bootstrap sample $\mathcal{D}^{(b)}$;
2. grow a tree on $\mathcal{D}^{(b)}$;
3. at each node, choose the split only from a random subset of m features rather than all p features.

For prediction, aggregate the trees using a majority vote, or by averaging their predicted class probabilities (soft voting).

Example 1.3: Medical triage and dominant variables

Suppose oxygen saturation is by far the strongest triage variable. Under plain bagging, many trees may repeatedly split first on that same variable. Under Random Forest, some trees are forced to consider other variables such as respiratory rate, reported symptom severity, or patient age. This increases diversity across trees and may improve the ensemble.

Remark 1.6: Interpretation of Random Forest

Random Forest is often the first strong ensemble baseline in practice because it is: accurate on many tabular datasets, relatively robust, less sensitive to tuning than many boosted methods, naturally suited to nonlinear interactions.

At the same time, Random Forests are less transparent than a single tree, and feature importance measures require careful interpretation, especially in the presence of correlated variables.

Remark 1.7: Transparency and feature importance in Random Forests

A single decision tree is transparent because one can literally read its splits from top to bottom and explain a prediction as a path such as “if age > 50 and cholesterol > 220, then class = risk.” A Random Forest is an average of many trees, often hundreds, so there is no single simple path that represents the whole model. This usually improves prediction, but it reduces interpretability.

Feature importance also requires care. In a Random Forest, an important variable is usually one that helps prediction a lot across the ensemble. But this becomes tricky when variables are correlated. For example, suppose both `age` and `years_since_retirement` carry almost the same information. The forest may use one of them in some trees and the other in other trees. Then the apparent importance may be split between them, or one may look unimportant only because its correlated partner was chosen first. Thus, “low importance” does not always mean “no information,” and “high importance” does not always mean “unique causal role.”

1.7 Measuring Feature Importance

Because we cannot trace a single decision path in a Random Forest, feature importance is usually assessed by measuring how much each variable contributes to the predictive performance of the ensemble. Two common methods are:

- **Impurity-based importance (Gini importance):** This sums the reduction in node impurity (such as the Gini index or entropy) achieved by a feature across all trees. It is fast to compute, but it can be misleading when predictors are highly correlated or when some variables have many possible split points or many categories.
- **Permutation importance:** This measures how much predictive performance worsens when the values of one feature are randomly shuffled in validation, test, or out-of-bag data. A large drop in performance suggests that the model relied strongly on that feature.

When predictors are strongly correlated, feature importance should be interpreted with caution, because correlated variables may share predictive information. In such cases, it is often useful to examine correlated features together, or to use grouped permutation importance, rather than assuming that one variable alone carries the entire importance.

1.8 Boosting: Sequential Error Correction

Bagging builds trees independently and then aggregates them. Boosting is fundamentally different. It builds trees one after another, and each new tree is chosen specifically to improve the current ensemble.

Definition 1.7: Boosting

Boosting is an ensemble strategy in which base learners are added sequentially, each one chosen to improve the performance of the current ensemble.

The core idea is:

Do not try to learn one perfect tree. Learn a sequence of small trees, where each new tree corrects part of what the current ensemble still gets wrong.

If $h_t(x)$ denotes the prediction of the t -th tree, a boosted model often has the additive form

$$F(x) = \sum_{t=1}^T \eta h_t(x),$$

where

- $h_t(x)$ is the prediction of the t -th tree,
- $\eta > 0$ is the learning rate,
- T is the number of boosting rounds.

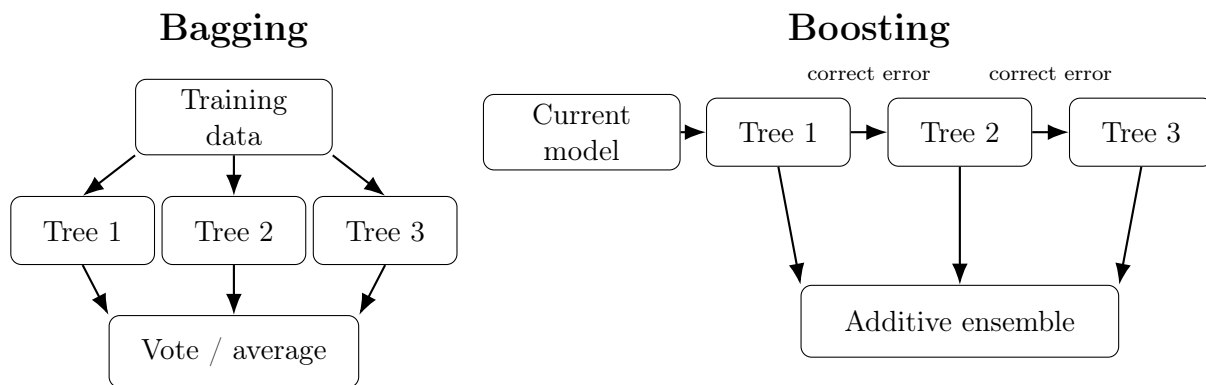


Figure 1: Bagging builds trees in parallel and aggregates them. Boosting builds trees sequentially, where each new tree improves the current ensemble.

There are two useful interpretations:

- **Error-focused view** Early boosting methods are often explained as increasing the influence of misclassified examples. Hard cases receive more attention from later learners.
- **Residual or gradient view.** Modern gradient boosting generalizes this idea. At each step, the algorithm computes how the current model is wrong according to a chosen loss function, then fits the next tree to that error signal.

The second view is more general and leads directly to modern boosted tree systems.

1.9 AdaBoost

AdaBoost is the classical error-focused boosting method. Its main idea is simple: observations that are hard to classify receive more weight in later rounds, so that the next weak learner pays more attention to them.

In practice, the weak learner is often a very small tree, for example a decision stump, that is, a tree with only one split.

In its classical form, AdaBoost is often written with labels coded as $y_i \in \{-1, +1\}$. This is only a coding convention; the conceptual idea is still binary classification.

Algorithm 1.3: AdaBoost for binary classification

Start with equal observation weights:

$$w_i^{(1)} = \frac{1}{n}, \quad i = 1, \dots, n.$$

For $t = 1, \dots, T$:

1. fit a weak classifier h_t using the current observation weights, so that observations with larger weights matter more in training;
2. compute its weighted error:

$$\text{err}_t = \sum_{i=1}^n w_i^{(t)} \mathbf{1}\{h_t(x_i) \neq y_i\};$$

3. assign the learner weight:

$$\alpha_t = \frac{1}{2} \log \frac{1 - \text{err}_t}{\text{err}_t};$$

4. update and normalize the observation weights:

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))}{\sum_{j=1}^n w_j^{(t)} \exp(-\alpha_t y_j h_t(x_j))}.$$

The final classifier is obtained by a weighted vote of the weak learners:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right).$$

Thus, weak learners with smaller weighted error receive larger weights and have more influence on the final prediction.

Remark 1.8: Interpreting the learner weight α_t

The quantity α_t determines how much influence the t -th weak learner has in the final weighted vote. If

$$0 < \text{err}_t < 0.5,$$

then $\alpha_t > 0$, so the learner receives a positive weight. If

$$\text{err}_t = 0.5,$$

then $\alpha_t = 0$, so the learner contributes nothing to the final classifier.

If $\text{err}_t = 0$ (respectively, $\text{err}_t = 1$), then the weak learner classified all weighted observations

correctly (respectively, incorrectly), and the formula for α_t is not finite. In practice, this corresponds to a degenerate case, and one usually stops early or handles it separately. Thus, AdaBoost is intended for weak learners that perform at least slightly better than random guessing, typically with

$$0 < \text{err}_t < 0.5.$$

Remark 1.9: How Step 4 changes the observation weights

Since both y_i and $h_t(x_i)$ take values in $\{-1, +1\}$, the condition

$$h_t(x_i) = y_i$$

means that observation i is classified correctly, while

$$h_t(x_i) \neq y_i$$

means that it is classified incorrectly.

Step 4 increases the influence of misclassified observations and decreases the influence of correctly classified ones. Before normalization, the update can be written as

$$w_i^{(t+1)} \propto \begin{cases} w_i^{(t)} e^{-\alpha_t}, & \text{if } h_t(x_i) = y_i, \\ w_i^{(t)} e^{\alpha_t}, & \text{if } h_t(x_i) \neq y_i. \end{cases}$$

Thus, correctly classified observations are downweighted, while misclassified observations are upweighted. After that, the weights are normalized so that they sum to 1.

Insight 1.3: AdaBoost versus bagging

Bagging builds many trees independently and stabilizes prediction by aggregation. AdaBoost builds learners sequentially and changes the observation weights from round to round so that later learners focus more on cases that were previously misclassified.

Warning 1.3: A practical caution

Because AdaBoost repeatedly increases the influence of observations that were misclassified in earlier rounds, it can over-focus on mislabeled cases, outliers, or strong noise. In such situations, the algorithm may keep trying to correct observations that should not be fitted closely, which can hurt generalization. This sensitivity is a general property of the method. In modern tabular-data applications, AdaBoost is often less preferred, because stronger and more flexible boosted-tree methods are available.

1.10 Gradient Boosting for Classification

1.10.1 From scores to probabilities

In binary classification, gradient boosting usually models a *score function* $F(x)$ rather than a class label directly. This fits naturally with what we already know from logistic regression: a score may be transformed into a probability through the logistic function.

Insight 1.4: Score function and probability

In binary classification, a model may first produce a score $F(x)$ and then convert it to a probability by

$$p(x) = \sigma(F(x)) = \frac{1}{1 + e^{-F(x)}}.$$

If $F(x)$ is interpreted on a log-odds-like scale, then the logistic function turns that score into a probability between 0 and 1.

A class label is then obtained only after choosing a threshold. If the threshold is 0.5, this corresponds to predicting class 1 when the estimated probability is at least 0.5. But as discussed in the earlier lecture on evaluation and cost-sensitive classification, the threshold should depend on the application, the class distribution, and the cost of errors.

Remark 1.10: Connection to logistic regression

The connection to logistic regression is conceptual rather than identical in form. In both settings, one can think in terms of a score that is converted to a probability. But logistic regression models one global linear relation, while gradient boosting builds an additive sequence of trees that can capture nonlinearities and interactions.

1.10.2 Generic gradient-boosting algorithm

Let $L(y, s)$ be a loss function, differentiable with respect to s . Gradient boosting generalizes the error-correction idea by using the loss function to define what the next correction should be.

Algorithm 1.4: Generic gradient boosting

Let $L(y, s)$ be a loss function, differentiable with respect to s .

Initialize the model with a simple constant score function:

$$F_0(x) = c, \quad \text{where } c = \arg \min_u \sum_{i=1}^n L(y_i, u).$$

Choose the learning rate $\eta > 0$ and the number of boosting rounds T .

For $t = 1, \dots, T$:

1. for each training observation x_i , compute the pseudo-residual

$$r_{it} = - \left. \frac{\partial L(y_i, s)}{\partial s} \right|_{s=F_{t-1}(x_i)};$$

2. fit a regression tree $h_t(x)$ to the targets r_{it} , including appropriate numerical values in its leaves;
3. update the model score function:

$$F_t(x) = F_{t-1}(x) + \eta h_t(x).$$

After T rounds, use the final score $F_T(x)$ to produce a probability or a class label.

Remark 1.11: Meaning of the notation in the algorithm

In the algorithm above:

- (x_i, y_i) is the i -th training observation and its class label;

- $L(y, s)$ is the loss function, differentiable with respect to s ;
- η is the learning rate;
- T is the total number of boosting rounds;
- $F_0(x)$ is the initial score function;
- $F_t(x)$ is the updated score function after round t .
- r_{it} is the pseudo-residual or negative-gradient signal;
- $h_t(x)$ is the regression tree fitted at round t .

Definition 1.8: Regression tree in boosting

A *regression tree* is a decision tree whose leaves contain numerical values rather than class labels. In gradient boosting, these leaf values represent corrections that should be added to the current score.

Example 1.4: A small correction tree

A regression tree added during boosting might output:

- -0.3 if oxygen saturation is high and symptom severity is mild;
- $+0.4$ if oxygen saturation is low and respiratory rate is high;
- $+0.1$ in another region.

This tree is not yet a final classifier. It is a rule for adjusting the current score upward or downward in different regions of the feature space.

1.10.3 How to understand the gradient boosting update

Gradient boosting does not build one final classifier in a single step. Instead, it builds the model gradually by adding small correction trees.

At boosting round t , the current model is

$$F_{t-1}(x).$$

This is a score function: for each observation x_i , it produces a current score $F_{t-1}(x_i)$. In binary classification, this score is often converted into a probability only at the end, for example through the logistic function.

For the i -th observation, the loss is written as

$$L(y_i, F_{t-1}(x_i)),$$

where y_i is the true class label and $F_{t-1}(x_i)$ is the current prediction score.

When we compute

$$\left. \frac{\partial L(y_i, s)}{\partial s} \right|_{s=F_{t-1}(x_i)}$$

we differentiate with respect to s and evaluate the derivative at the *current model output* $F_{t-1}(x_i)$, while treating the true class label y_i as fixed. Thus the derivative answers the question:

If we change the current score a little, how will the loss change?

If the derivative is positive, increasing the score would increase the loss, so the score should move downward; if the derivative is negative, increasing the score would decrease the loss, so the

score should move upward. Therefore, the negative gradient points in the direction in which the current score should move in order to reduce the loss.

These derivatives are known only at the training points, and we need a function $h_t(x)$ that generalizes these desired corrections over the feature space. This is why $r_{it} = - \left. \frac{\partial L(y_i, s)}{\partial s} \right|_{s=F_{t-1}(x_i)}$ is used as the target in round t . To fit a regression tree $h_t(x)$ to the targets r_{it} means to use the predictors x_i as inputs and the pseudo-residuals r_{it} as numerical response values. Since the pseudo-residuals represent the direction and size of the change needed in the current score, the fitted tree learns a function that approximates these corrections in different regions of the feature space i.e. it learns a correction rule. Thus, for different future values of the predictors x , the tree can output an estimated correction of the current score.

After fitting this regression tree $h_t(x)$, the model is updated by

$$F_t(x) = F_{t-1}(x) + \eta h_t(x),$$

where $\eta > 0$ is the learning rate. Thus, the new tree does not replace the existing model. Instead, it adds a small correction to it.

So the overall logic of gradient boosting is:

1. start with a simple initial score;
2. measure how the current scores should change to reduce the loss;
3. fit a small regression tree to these desired corrections;
4. add that tree, scaled by the learning rate, to the current model;
5. repeat many times.

Only after all boosting rounds are completed is the final score $F_T(x)$ converted into a probability or a class decision.

1.10.4 Example with squared-error loss

We use squared-error loss in the following example because it makes the update steps especially easy to see. This is not the most standard loss for classification boosting; in practice, binary classification is usually handled with a classification-oriented loss such as logistic loss. However, the basic correction logic is the same.

Example 1.5: Several boosting steps with squared-error loss

Consider this tiny dataset

Observation	x	y
1	1	0
2	2	0
3	3	0
4	4	1
5	5	1

Suppose squared-error loss is used:

$$L(y, s) = \frac{1}{2}(y - s)^2,$$

The partial derivative of this function with respect to s is $s - y$. At boosting round t , we

evaluate the negative gradient at $y = y_i$ and $s = F_{t-1}(x_i)$, where y_i is the true class label and $F_{t-1}(x_i)$ is the current prediction score.

Choose a learning rate η , for example $\eta = 0.1$ and the number of boosting rounds T .

Initialize. Under squared-error loss, the constant value that minimizes the total loss

$$\sum_{i=1}^n \frac{1}{2}(y_i - c)^2$$

is the sample mean of the y_i 's.

Thus we initialize the model with constant score function equal to the mean of the class labels:

$$F_0(x) = \frac{0 + 0 + 0 + 1 + 1}{5} = 0.4.$$

So initially all five observations receive score 0.4.

Step 1: Compute pseudo-residuals. The negative gradient is

$$r_{i1} = y_i - F_0(x_i).$$

Thus,

$$r_{11} = -0.4, \quad r_{21} = -0.4, \quad r_{31} = -0.4, \quad r_{41} = 0.6, \quad r_{51} = 0.6.$$

Step 2: Fit a regression tree to the pseudo-residuals. Among the candidate thresholds 1.5, 2.5, 3.5, and 4.5, the split at 3.5 is best, because it separates the pseudo-residuals into two perfectly homogeneous groups. Thus the one-split regression tree is

$$h_1(x) = \begin{cases} -0.4, & x < 3.5, \\ +0.6, & x \geq 3.5. \end{cases}$$

Step 3: Update the model. For the chosen learning rate $\eta = 0.1$ we have

$$F_1(x) = F_0(x) + 0.1 h_1(x).$$

So the new scores are

$$0.36, 0.36, 0.36, 0.46, 0.46.$$

Convert scores to class labels if needed. Using threshold 0.5, the predicted classes after the first boosting step are still

$$0, 0, 0, 0, 0.$$

So after one step, the scores have improved, but the class labels have not changed yet. If we continue boosting, the same split remains optimal and the scores continue moving toward the true labels. After the second boosting round, the scores become

$$0.324, 0.324, 0.324, 0.514, 0.514,$$

so the predicted classes become

$$0, 0, 0, 1, 1.$$

Thus, this example shows the real logic of gradient boosting: each round makes a small correction to the score function, and only after enough corrections may the final class labels change. In other words, scores may improve before the final class labels change. If we continue boosting with $\eta = 0.1$, the same split remains optimal and the scores keep moving toward the true labels. For example,

$$F_5(x) = (0.236196, 0.236196, 0.236196, 0.645706, 0.645706).$$

Thus, in this example, the class labels become correct after the second boosting round, while additional rounds mainly increase the confidence of the predictions. A larger learning rate would usually make this correction happen faster, but in general it can also lead to different intermediate scores and possibly different class labels.

Remark 1.12: Choosing the number of boosting rounds

There is no universal fixed number of boosting iterations that is best for all problems. In practice, one usually chooses the number of rounds by validation or cross-validation. The model is trained for many rounds, but performance is monitored on separate validation data, and one stops at the iteration where validation performance is best. This is called *early stopping*.

The number of boosting rounds and the learning rate must be chosen together: a smaller learning rate usually requires more iterations, while a larger learning rate often requires fewer.

1.10.5 Why gradient boosting is powerful and how it can overfit

Gradient boosting is powerful because:

- trees capture nonlinearities and interactions;
- boosting reduces the remaining error step by step;
- small learning steps can create a flexible yet controllable model;
- regularization and validation can be used to control complexity.

Remark 1.13: Regularization and validation in gradient boosting

Here, regularization means controlling model flexibility through shallow trees, small learning rates, subsampling, and limited boosting rounds, while validation means monitoring performance on separate data to choose hyperparameters and stop training appropriately.

At the same time, boosting can overfit when:

- too many trees are added;
- individual trees are too deep;
- the learning rate is too large;
- the validation protocol is poor;
- hyperparameters are tuned on the test set.

Warning 1.4: Boosting is powerful, but not immune to noise

A boosted model can fit subtle signal very well, but the same flexibility can also fit noise. Small learning rates, shallow trees, proper validation, and early stopping are therefore central practical safeguards.

2 Modern Boosted Tree Systems

XGBoost, LightGBM, and CatBoost are all modern implementations of boosted decision trees. They should not be understood as three unrelated classifiers. Their importance is practical: they keep the core boosting idea, but organize it differently in order to improve speed, robustness, regularization, or handling of categorical variables.

2.1 Common backbone

Their common backbone is:

1. start from a simple model;
2. repeatedly compute where the current model is wrong according to a loss function;
3. fit a new tree that improves that;
4. add the tree to the ensemble;
5. use regularization and validation to control complexity.

All three methods:

- build additive ensembles of trees;
- rely on greedy split selection inside each tree;
- aim for strong predictive performance on tabular data;
- require careful hyperparameter tuning and careful evaluation.

2.2 Design differences

It is helpful to think of the three systems as emphasizing different design goals:

- **XGBoost**: strong regularization and a robust general-purpose design;
- **LightGBM**: speed and scalability;
- **CatBoost**: principled handling of categorical variables.

Thus, the methods are not mainly distinguished by *what* they try to do, but by *how* they organize the boosting process and which practical problems they emphasize.

2.3 XGBoost

XGBoost is often treated as a strong general-purpose baseline for boosted trees. It emphasizes careful regularization, efficient implementation, and reliable performance on many tabular datasets.

Remark 2.1: How to think about XGBoost

The main conceptual picture of XGBoost is not “a completely different method,” but “a highly optimized, strongly regularized gradient-boosted tree system.”

In practice, XGBoost is often associated with:

- strong predictive performance;
- good control of model complexity through regularization;
- wide practical use and mature software support.

It is also useful to remember that XGBoost is more than a fast implementation of basic gradient boosting. It adds extra optimization and regularization ideas that often make training more stable and effective in practice.

Typical cautions include:

- careful tuning is still needed;
- categorical variables often require explicit preprocessing;
- feature importance should not be interpreted causally.

2.4 LightGBM

LightGBM is designed with computational speed and large-scale learning in mind. Two ideas are especially important for understanding it:

- **histogram-based splitting:** continuous feature values are grouped into bins, which makes split search much faster;
- **leaf-wise tree growth:** instead of growing all leaves level by level, the algorithm extends the leaf that currently looks most promising.

Remark 2.2: How to think about LightGBM

The key practical message is speed and scalability. LightGBM is especially attractive when the dataset is large and fast experimentation matters.

These design choices can make LightGBM very efficient, especially on large tabular datasets. At the same time, leaf-wise growth can produce more aggressive trees, so the method may need careful control of depth, leaf size, and regularization.

Typical strengths include:

- fast training;
- strong performance on large tabular datasets;
- efficient handling of many features.

Typical cautions include:

- aggressive tree growth can overfit if not controlled;
- hyperparameters controlling tree size and regularization matter strongly;

- speed does not remove the need for proper evaluation.

2.5 CatBoost

CatBoost is particularly important when categorical variables play a central role. It is designed to handle categorical information in a more principled way than naive one-hot encoding in many settings.

A central idea in CatBoost is the use of *ordered target statistics*. Instead of replacing a category by a naive target mean computed from the whole dataset, which may create target leakage, CatBoost uses an ordered scheme that reduces this problem and gives a more careful estimate of category information.

Remark 2.3: How to think about CatBoost

CatBoost is especially valuable when high-cardinality categorical variables are important, categories are rare, and careless encoding could create leakage or unstable estimates.

CatBoost is also known for using *oblivious* (symmetric) trees, where all nodes at the same depth use the same split condition.

Example 2.1: A simple categorical contrast

Suppose an admissions dataset contains a variable `high_school_ID` with many possible categories. A naive one-hot encoding may create a large number of binary columns and may behave poorly if some schools appear only a few times. CatBoost instead tries to summarize category signal through ordered target statistics, which can be more compact and often more robust in such situations.

Typical strengths include:

- strong handling of categorical predictors;
- reduced danger of naive target leakage in category encoding;
- good performance on tabular problems where categories are central.

Typical cautions include:

- it is not automatically best on every dataset;
- careful validation and tuning are still needed;
- interpretability issues remain similar to those of other boosted-tree systems.

Insight 2.1: Main takeaway of this section

XGBoost, LightGBM, and CatBoost are all boosted-tree systems built on the same general idea. Their practical differences come mainly from design priorities: XGBoost emphasizes robust regularized boosting, LightGBM emphasizes speed and scalability, and CatBoost emphasizes principled handling of categorical variables.

2.6 Practical Hyperparameters and Their Meaning

Although software packages differ, the most important conceptual hyperparameters are similar across modern boosted-tree systems.

- **Number of trees / boosting rounds.** More trees increase expressive power, because the model can make more correction steps. But too many trees may overfit, especially if the learning rate is large or the individual trees are too complex.
- **Learning rate.** The learning rate controls how strongly each new tree changes the current score function. Smaller learning rates usually require more boosting rounds, but they often give a more gradual and stable fit.
- **Tree depth or number of leaves.** These parameters control the complexity of each individual tree. Shallow trees produce simpler corrections, while deeper trees or larger numbers of leaves can capture richer interactions but may overfit more easily.
- **Subsampling.** Using only a subset of rows or columns at each boosting round can regularize the model and reduce variance.
- **Regularization parameters.** Modern implementations often include penalties or constraints that discourage overly complex trees or overly large leaf values.

Remark 2.4: Three high-impact hyperparameters

Among the most influential tuning parameters in boosted trees are:

- the *number of trees*,
- the *learning rate*,
- the *complexity of each tree*.

These parameters interact strongly and should usually be tuned together rather than one by one in isolation.

Remark 2.5: A practical view of fine-tuning

Fine-tuning should usually begin with the three high-impact hyperparameters above. A smaller learning rate often requires more trees. Larger or deeper trees often require stronger regularization or fewer rounds. In practice, tuning is usually done with validation or cross-validation, often together with early stopping, rather than by trying to optimize one parameter at a time on the test set.

2.7 Practical Do's and Don'ts for Modern Boosted Tree Systems

Do's

- Begin with a simpler baseline before moving to a more elaborate boosted system.
- Use validation, cross-validation, and early stopping whenever possible.
- Tune the learning rate, number of trees, and tree complexity jointly.
- Check carefully how categorical variables are handled in the chosen implementation.
- Interpret feature importance as predictive importance, not as causal importance.

Don'ts

- Do not judge model quality from training error alone.
- Do not tune hyperparameters on the test set.

- Do not assume that a modern library automatically removes the need for careful preprocessing and validation.
- Do not treat one package as universally best for every tabular dataset.
- Do not ignore leakage, mislabeled data, or unstable category encoding.

XGBoost, LightGBM, and CatBoost are all members of the same boosted-tree family. Their main differences are practical rather than conceptual in the deepest sense. XGBoost emphasizes robust regularized boosting, LightGBM emphasizes speed and scalability, and CatBoost emphasizes principled handling of categorical variables.

For practical use, you should remember that the most important tuning decisions usually concern the number of trees, the learning rate, and the complexity of the individual trees. These choices interact strongly and should be made with validation rather than by trial and error on the test set.

2.8 Compact Comparison of Tree-Based Classification Methods

Method	Parallel / sequential	Main idea	Main effect	Typical use / caution
Single tree	Single model	Greedy partition of the feature space	Interpretable local rules	Easy to explain, but unstable and often weaker predictively
Bagging	Parallel	Bootstrap many trees and aggregate them	Mainly variance reduction	Useful when trees are unstable; does not solve leakage or interpretability
Random Forest	Parallel	Bagging + random feature subsets	Stronger variance reduction through decorrelation	Strong baseline for tabular data; less transparent than one tree
AdaBoost	Sequential	Reweight harder observations	Sequential emphasis on misclassified cases	Conceptually important; can be sensitive to noisy labels
Gradient boosting	Sequential	Fit correction trees to a loss-based error signal	Flexible additive improvement	Very powerful, but can overfit without validation and regularization
XGBoost / LightGBM / CatBoost	Sequential	Modern boosted-tree systems with different design emphases	High predictive performance with practical optimizations	Choice depends on scale, regularization needs, and categorical-feature structure

2.9 Summary

A single decision tree is attractive because it is easy to interpret, but it may be unstable and sensitive to the particular training sample. Ensemble methods address this weakness in two main ways.

Bagging and Random Forest mainly reduce variance by aggregating many trees. Random Forest strengthens this idea by forcing more diversity through feature randomness. Boosting is different: it builds trees sequentially so that each new tree improves the current ensemble. AdaBoost

emphasizes harder cases through reweighting, while gradient boosting fits correction trees to a loss-based error signal.

Modern systems such as XGBoost, LightGBM, and CatBoost are all boosted-tree systems built on that same general idea, but with different practical design priorities. Their choice depends less on abstract theory alone and more on issues such as regularization, computational scale, and handling of categorical variables.

The main practical lesson of this lecture is that powerful ensemble methods are not automatic solutions. Their performance depends on thoughtful tuning, honest validation, and a clear understanding of what kind of prediction problem is actually being solved.